

# LLM Book Pipeline

---

*Technical Report: Architecture, Design, and Analysis*

**Project:** llm-book-pipeline

**Description:** Automated YouTube-to-eBook PDF pipeline powered by LLMs

**Source:** <https://github.com/vaibhav3000/llm-book-pipeline>

Prepared by: **Vaibhav Mahore**

Indian Institute of Science, Bangalore

B.Tech - Mathematics and Computing

Version 1.0 | April 23, 2026

---

## Abstract

This report presents a comprehensive technical analysis of **llm-book-pipeline**, an automated four-stage pipeline that converts a YouTube playlist of educational lectures into an eBook-quality PDF manuscript using Large Language Model (LLM) APIs. The pipeline ingests a YouTube playlist (specifically the “Building LLMs from Scratch” series), fetches and cleans raw transcripts via `youtube-transcript-api` and `yt-dlp`, chunks the transcript text using a sliding-window strategy with configurable overlap, expands each chunk into polished book-quality prose using an LLM (DeepSeek/Groq/Anthropic), assembles a complete manuscript with AI-generated foreword and glossary, and renders a publication-quality PDF via ReportLab—all without any LaTeX dependency. The system is designed for idempotency, full resumability, and cost efficiency through file-based caching of all LLM outputs. A free-API variant using the Groq API (Llama 3.3-70B) is also provided. This report covers the problem statement, end-to-end architecture, module-level code analysis, algorithmic design decisions, LLM integration strategy, prompt engineering methodology, known limitations, and potential improvements.

## 0 Contents

---

|          |                                                                                   |          |
|----------|-----------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                               | <b>2</b> |
| 1.1      | Motivation . . . . .                                                              | 2        |
| 1.2      | Goals . . . . .                                                                   | 3        |
| <b>2</b> | <b>Problem Statement</b>                                                          | <b>3</b> |
| 2.1      | Formal Definition . . . . .                                                       | 3        |
| 2.2      | Constraints . . . . .                                                             | 4        |
| <b>3</b> | <b>System Architecture</b>                                                        | <b>4</b> |
| 3.1      | High-Level Pipeline . . . . .                                                     | 4        |
| 3.2      | Repository Structure . . . . .                                                    | 4        |
| 3.3      | Configuration System . . . . .                                                    | 5        |
| <b>4</b> | <b>Detailed Pipeline Explanation</b>                                              | <b>5</b> |
| 4.1      | Stage 1: Transcript Fetching ( <code>01_fetch_transcripts.py</code> ) . . . . .   | 5        |
| 4.1.1    | Objective . . . . .                                                               | 5        |
| 4.1.2    | Key Functions . . . . .                                                           | 5        |
| 4.1.3    | Output Schema . . . . .                                                           | 6        |
| 4.1.4    | Idempotency . . . . .                                                             | 6        |
| 4.2      | Stage 2: Chapter Writing ( <code>02_write_chapters.py</code> ) . . . . .          | 6        |
| 4.2.1    | Objective . . . . .                                                               | 6        |
| 4.2.2    | Chunking Strategy . . . . .                                                       | 6        |
| 4.2.3    | Prompt Engineering . . . . .                                                      | 6        |
| 4.2.4    | Retry / Rate-Limit Logic . . . . .                                                | 7        |
| 4.2.5    | API Abstraction . . . . .                                                         | 7        |
| 4.3      | Stage 3: Manuscript Assembly ( <code>03_assemble_manuscript.py</code> ) . . . . . | 7        |
| 4.3.1    | Objective . . . . .                                                               | 7        |
| 4.3.2    | Generated Components . . . . .                                                    | 7        |
| 4.3.3    | Manuscript Structure . . . . .                                                    | 8        |
| 4.4      | Stage 4: PDF Rendering ( <code>04_render_pdf.py</code> ) . . . . .                | 8        |
| 4.4.1    | Objective . . . . .                                                               | 8        |
| 4.4.2    | Architecture . . . . .                                                            | 8        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 4.4.3     | Inline Markdown Conversion                        | 8         |
| 4.4.4     | Cover Page Design                                 | 8         |
| 4.4.5     | Page Headers and Footers                          | 8         |
| <b>5</b>  | <b>Module-Level File Analysis</b>                 | <b>9</b>  |
| 5.1       | build_book.py — Orchestrator                      | 9         |
| 5.2       | config.yaml — Configuration                       | 9         |
| 5.3       | requirements.txt — Dependencies                   | 9         |
| 5.4       | patch.py and patch2.py — API Migration Scripts    | 9         |
| <b>6</b>  | <b>LLM Integration Analysis</b>                   | <b>10</b> |
| 6.1       | Supported Backends                                | 10        |
| 6.2       | Where the LLM is Used                             | 10        |
| 6.3       | Cost Estimation                                   | 10        |
| 6.4       | Grounding Strategy                                | 10        |
| <b>7</b>  | <b>Algorithms and Logic</b>                       | <b>10</b> |
| 7.1       | Sliding-Window Chunking                           | 10        |
| 7.2       | Exponential Backoff with Groq Rate-Limit Parsing  | 11        |
| 7.3       | Markdown-to-ReportLab Parser                      | 11        |
| <b>8</b>  | <b>Design Patterns and Architecture Decisions</b> | <b>11</b> |
| <b>9</b>  | <b>Challenges and Limitations</b>                 | <b>12</b> |
| 9.1       | Known Weaknesses                                  | 12        |
| 9.2       | Edge Cases                                        | 12        |
| 9.3       | Failure Scenarios                                 | 12        |
| <b>10</b> | <b>Future Improvements</b>                        | <b>13</b> |
| <b>11</b> | <b>Results and Output</b>                         | <b>13</b> |
| <b>12</b> | <b>Conclusion</b>                                 | <b>14</b> |

## 1 Introduction

---

The proliferation of technical educational content on YouTube has created a large body of unstructured spoken-word knowledge. While video playlists are an effective medium for teaching, they lack the searchability, portability, and depth of well-edited written books. This project bridges that gap: given any YouTube playlist, the pipeline automatically produces a complete, formatted book in PDF format, with content expanded and enriched by a Large Language Model.

The target playlist—“Building LLMs from Scratch” by Dr. Raj Dander (VIUA)—contains 43 videos covering the complete construction of a transformer-based LLM from first principles, based on Sebastian Raschka’s book. The pipeline converts this into a multi-chapter book, generating over 100,000 words of polished technical prose.

### 1.1 Motivation

Several educational pipelines exist for text summarization, but **expansion** of sparse spoken transcripts into comprehensive book chapters is a substantially harder problem. Key challenges include:

- Transcripts are noisy: they lack punctuation, contain filler words, music annotations, and speaker artifacts.
- Transcripts are sparse: a 20-minute lecture yields perhaps 3,000 words, but a well-written book chapter on the same topic might be 5,000-8,000 words.
- LLM context windows are finite: a full transcript may exceed the model's input window, necessitating chunking.
- API costs must be minimized: re-running the LLM on previously-processed content would be wasteful and expensive.
- The final output must be typeset: raw Markdown is insufficient for a publication-quality deliverable.

## 1.2 Goals

1. **Reproducibility:** Every LLM output is cached to disk; re-runs produce identical results.
2. **Resumability:** Each stage is idempotent; the pipeline can resume from any intermediate point.
3. **Grounded generation:** The LLM is explicitly instructed not to hallucinate facts outside the provided transcript.
4. **Zero hallucination risk:** A strict system prompt enforces source-only generation.
5. **Portability:** No LaTeX or Pandoc dependency; the PDF is generated entirely in Python via ReportLab.
6. **Multi-API support:** Works with DeepSeek, Anthropic Claude, Groq Llama 3, and Google Gemini.

## 2 Problem Statement

---

### 2.1 Formal Definition

Let  $V = \{v_1, v_2, \dots, v_n\}$  be an ordered sequence of YouTube videos forming a playlist. Each video  $v_i$  has an associated spoken-word transcript  $T_i$ , which is a time-stamped sequence of text segments:

$$T_i = \{(w_{i,1}, t_{i,1}, d_{i,1}), (w_{i,2}, t_{i,2}, d_{i,2}), \dots\}$$

where  $w_{i,j}$  is the text of segment  $j$ ,  $t_{i,j}$  is its start time (seconds), and  $d_{i,j}$  is its duration.

The goal is to produce a structured document  $D$  such that:

$$D = \mathcal{P}\left(\mathcal{A}(\{C_i\}_{i=1}^n)\right), \quad \text{where } C_i = \mathcal{L}(\mathcal{F}(T_i))$$

Here:

- $\mathcal{F}$  is the **cleaning and chunking** function, mapping raw transcript  $T_i$  to a set of text chunks  $\{c_{i,1}, \dots, c_{i,k}\}$ .
- $\mathcal{L}$  is the **LLM expansion** function, mapping chunks to a polished prose chapter  $C_i$ .
- $\mathcal{A}$  is the **assembly** function, combining chapters with generated foreword, TOC, and glossary.
- $\mathcal{P}$  is the **PDF rendering** function producing the final output document.

## 2.2 Constraints

### Key Design Constraints

- LLM context limit:  $\max(\text{chunk size}) \leq C_{\max}$  (typically 3,000–4,096 tokens).
- No hallucination:  $\mathcal{L}(c) \subseteq \text{semantic span}(c)$  – all generated content grounded in source.
- Idempotency: Running any stage twice must produce the same output if inputs are unchanged.
- Cost: total API cost  $\propto \sum_i [|T_i| / (\text{chunk\_size} - \text{overlap})]$ .

## 3 System Architecture

### 3.1 High-Level Pipeline

The system is organized as a linear four-stage pipeline. Stages communicate exclusively through the filesystem, making each stage independently runnable and testable.

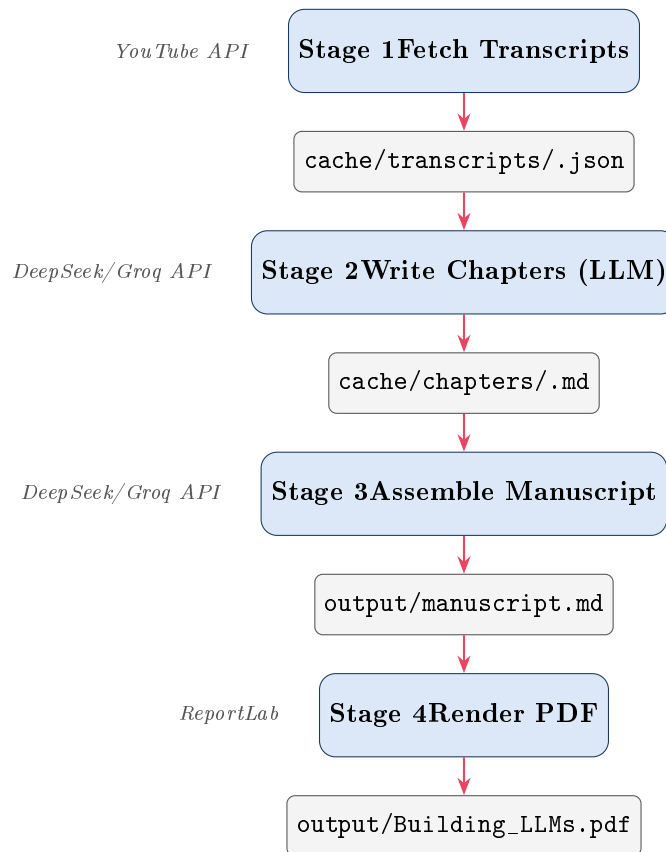


Figure 1: End-to-end pipeline architecture. Each stage reads from and writes to disk, enabling full resumability.

### 3.2 Repository Structure

#### Directory Layout

```

llm-book-pipeline/
|-- build_book.py          # Entrypoint: orchestrates all stages
|-- config.yaml           # All tunable parameters
|-- requirements.txt       # Python dependencies
|-- patch.py              # Gemini API migration patch

```

```

|-- patch2.py                # Gemini v2 API migration patch
|-- pipeline/
|   |-- 01_fetch_transcripts.py # Stage 1
|   |-- 02_write_chapters.py   # Stage 2 (LLM-heavy)
|   |-- 03_assemble_manuscript.py # Stage 3
|   '-- 04_render_pdf.py       # Stage 4
|-- cache/
|   |-- video_index.json       # Playlist metadata
|   |-- transcripts/           # <video_id>.json per video
|   '-- chapters/              # <video_id>.md per video
|-- output/
|   |-- manuscript.md          # Assembled full Markdown
|   '-- Building_LLMs_from_Scratch.pdf
'-- free_api_version/          # Groq (free-tier) variant
    |-- build_book.py
    |-- config.yaml
    '-- pipeline/              # Identical structure, Groq backend

```

### 3.3 Configuration System

All tunable parameters are centralized in `config.yaml`:

| Parameter                           | Default                    | Purpose                            |
|-------------------------------------|----------------------------|------------------------------------|
| <code>llm.model</code>              | <code>deepseek-chat</code> | LLM model identifier               |
| <code>llm.chunk_size</code>         | <code>3000</code>          | Characters per transcript chunk    |
| <code>llm.chunk_overlap</code>      | <code>200</code>           | Overlap between consecutive chunks |
| <code>llm.max_tokens</code>         | <code>4096</code>          | Max tokens in LLM response         |
| <code>pipeline.force_refetch</code> | <code>false</code>         | Override transcript cache          |
| <code>pipeline.force_rewrite</code> | <code>false</code>         | Override chapter cache             |
| <code>pipeline.max_videos</code>    | <code>null</code>          | Limit for testing                  |

## 4 Detailed Pipeline Explanation

### 4.1 Stage 1: Transcript Fetching (`01_fetch_transcripts.py`)

#### 4.1.1 Objective

Retrieve all video metadata from the YouTube playlist and download the auto-generated or manual caption transcript for each video. Save raw segments and cleaned text to disk.

#### 4.1.2 Key Functions

`get_playlist_videos(playlist_id)` Uses `yt-dlp` with `extract_flat=True` to retrieve the full video list without downloading any media. Returns a list of `{id, title, index}` dicts.

`fetch_transcript(video_id)` Calls `YouTubeTranscriptApi().fetch(video_id)` to retrieve time-stamped caption segments. Returns a list of `{text, start, duration}` dicts.

`clean_transcript(segments)` Joins all segment texts, then applies three regex operations:

1. Remove noise annotations: `re.sub(r'\[.*?\]', '', raw)`
2. Remove parenthetical artifacts: `re.sub(r'\(.*?\)', '', raw)`
3. Collapse whitespace: `re.sub(r'\s+', ' ', raw)`

#### 4.1.3 Output Schema

Each `cache/transcripts/<video_id>.json` contains:

```

1 {
2   "video_id": "Xpr8D6LeAtw",
3   "title": "Lecture 1: Building LLMs from Scratch...",
4   "index": 1,
5   "raw_segments": [{"text": "hello everyone", "start": 0.0, "duration": 1.2}, ...],
6   "cleaned_text": "hello everyone welcome to the introductory ...",
7   "word_count": 2623
8 }
```

#### 4.1.4 Idempotency

If a transcript file already exists and `force_refetch: false`, the stage prints `CACHED` and skips the video. A 0.5-second sleep between API calls implements polite rate limiting.

## 4.2 Stage 2: Chapter Writing (`02_write_chapters.py`)

### 4.2.1 Objective

This is the core LLM-heavy stage. It takes each cleaned transcript, divides it into overlapping chunks, and calls an LLM API to expand each chunk into polished book-quality prose.

### 4.2.2 Chunking Strategy

The sliding-window chunking algorithm is defined as follows. Let  $S$  be the cleaned transcript string of total length  $|S|$ , chunk size  $C$ , and overlap  $O$ :

$$\text{chunk}_k = S[s_k : \min(s_k + C, |S|)] \quad (1)$$

$$s_{k+1} = s_k + C - O \quad (2)$$

The process terminates when  $s_k \geq |S|$ . Chunks shorter than 100 characters are discarded. With the default values  $C = 3000$  and  $O = 200$ , the effective stride is 2800 characters, and adjacent chunks share a 200-character context window. This overlap ensures that ideas spanning a chunk boundary are not truncated mid-thought.

#### Why Overlap Matters

Without overlap, a sentence beginning at character 2990 of a chunk would be cut off. The 200-character overlap ensures the LLM sees sufficient trailing context from the previous chunk, producing more coherent transitions in the generated prose.

### 4.2.3 Prompt Engineering

Two prompt functions are used:

1. **Chapter Intro** (`write_chapter_intro`): A short, focused prompt using the first 2,000 characters of the transcript to generate a 2–3 sentence engaging chapter opening.

**2. Chunk Expansion (expand\_chunk):** Uses a full system prompt + user message. The system prompt enforces seven strict rules:

#### System Prompt Rules (verbatim)

1. ONLY use information from the provided transcript. Do NOT add outside facts.
2. Write in clear, flowing prose — no bullet summaries.
3. Expand on ideas: add transitions, explanations, implied examples.
4. Preserve all technical accuracy. Never paraphrase code or equations loosely.
5. Write 400–600 words per transcript excerpt chunk.
6. Maintain a consistent tone: technical yet accessible, like a well-written O'Reilly book.
7. Do NOT start with “In this section” or “In this chapter”. Just dive in.

The user message embeds chunk position context (**chunk K of N**), instructing the model to write an opening paragraph for chunk 1, a continuation for middle chunks, and a closing/transitional paragraph for the final chunk.

#### 4.2.4 Retry / Rate-Limit Logic

The `robust_api_call` function wraps every API call in an infinite retry loop with exponential backoff:

$$t_{\text{sleep}}(r) = \min(300, 2^r) \text{ seconds}$$

where  $r$  is the retry count. In the free-API version (Groq), the error message is parsed with a regex to extract the exact cooldown duration:

```
1 re.search(r'try again in (?:\d+)h?(?:\d+)m?([\d\.]+)s', err_msg)
```

If matched, the sleep duration is set to  $3600h + 60m + s + 5$  seconds (5-second buffer). This enables fully unattended overnight execution without manual intervention.

#### 4.2.5 API Abstraction

The OpenAI Python SDK's compatibility layer is exploited to call DeepSeek and Groq with identical code:

```
1 client = OpenAI(  
2     api_key=os.environ["DEEPSEEK_API_KEY"],  
3     base_url="https://api.deepseek.com"  
4 )
```

The same code works for Groq by changing `base_url` to `https://api.groq.com/openai/v1`.

### 4.3 Stage 3: Manuscript Assembly (03\_assemble\_manuscript.py)

#### 4.3.1 Objective

Combine all individual chapter files into a single coherent manuscript, and generate AI-authored front and back matter (foreword, table of contents, glossary).

#### 4.3.2 Generated Components

**Foreword** Generated by prompting the LLM with the list of chapter titles and requesting a 3–4 paragraph motivational foreword. Explicitly instructed to avoid fabricating author names, dates, or specific claims.

**Table of Contents** Built programmatically (no LLM call) from the ordered `video_index.json`.



**Glossary** The first 8,000 characters of the assembled content are sent to the LLM with a request to extract and define 15–20 key technical terms in **\*\*Term\*\***: **Definition** format.

### 4.3.3 Manuscript Structure

The final `manuscript.md` is structured as:

$$\underbrace{\text{Title Block}}_{\text{H1}} \rightarrow \text{TOC} \rightarrow \text{Foreword} \rightarrow \underbrace{C_1, C_2, \dots, C_n}_{\text{H2 per chapter}} \rightarrow \text{Glossary}$$

## 4.4 Stage 4: PDF Rendering (`04_render_pdf.py`)

### 4.4.1 Objective

Convert the assembled `manuscript.md` into a professional, typeset PDF using ReportLab’s Platypus framework. No LaTeX or Pandoc dependency.

### 4.4.2 Architecture

The renderer implements a custom Markdown parser that converts Markdown AST nodes to ReportLab *Flowables* — the fundamental rendering primitives:

| Markdown Token   | ReportLab Flowable     | Style                             |
|------------------|------------------------|-----------------------------------|
| # H1 (title)     | Paragraph              | BookTitle (32pt, white, centered) |
| ## H2 (chapter)  | Paragraph + HRFlowable | ChapterHeading (24pt)             |
| ### H3 (section) | Paragraph              | SectionHeading (15pt)             |
| --               | HRFlowable             | 80% width, 0.5pt, gray            |
| “...”            | Paragraph              | Courier 9pt, code background      |
| - item           | Paragraph              | TOCEntry or TOCEntryChapter       |
| Body text        | Paragraph              | BodyText (11pt, justified)        |

### 4.4.3 Inline Markdown Conversion

The `md_to_para` function handles inline formatting by first escaping XML characters (&, <, >), then applying regex substitutions to convert Markdown syntax to ReportLab XML tags:

$$\text{***text***} \rightarrow \langle \mathbf{b} \rangle \langle \mathbf{i} \rangle \text{text} \langle \mathbf{i} \rangle \langle \mathbf{b} \rangle \quad (3)$$

$$\text{**text**} \rightarrow \langle \mathbf{b} \rangle \text{text} \langle \mathbf{b} \rangle \quad (4)$$

$$\text{*text*} \rightarrow \langle \mathbf{i} \rangle \text{text} \langle \mathbf{i} \rangle \quad (5)$$

$$\text{'code'} \rightarrow \langle \text{font name="Courier" size="9"} \rangle \text{code} \langle \text{font} \rangle \quad (6)$$

### 4.4.4 Cover Page Design

The cover page uses a `canvas` callback (`onFirstPage`) to draw a dark-navy background (`#1a1a2e`), a red accent bar, and a subtle grid pattern. The color palette mirrors a modern tech book aesthetic.

### 4.4.5 Page Headers and Footers

The `_header_footer` callback draws:

- A red rule line at top and bottom (0.5pt)

- Running header: “Building LLMs from Scratch”
- Centered page number (offset by 1 to exclude the cover page)

The first page (cover) is exempted via a `doc.page > 1` guard.

## 5 Module-Level File Analysis

### 5.1 `build_book.py` — Orchestrator

#### Purpose

Single entrypoint that accepts CLI flags and delegates to each pipeline stage by running them as subprocess OS commands via `os.system()`. Handles config mutation for the `-max-videos` flag by writing back to `config.yaml`.

Key design observation: each stage is invoked as a subprocess (`os.system(f"python {script}")`), not imported as a module. This provides strong isolation — a crash in any stage produces a non-zero exit code that terminates the orchestrator cleanly. The `-from N`, `-only N`, `-dry-run` flags give granular control.

#### Design Note

Using `os.system` rather than `subprocess.run` means that stdout/stderr of each stage is not captured—it streams directly to the terminal. This is intentional for user visibility, but prevents programmatic log capture.

### 5.2 `config.yaml` — Configuration

A flat YAML file serving as the single source of truth for all pipeline parameters. `build_book.py` mutates this file at runtime to set `max_videos`, which is a side-effectful design that could cause issues with concurrent runs.

### 5.3 `requirements.txt` — Dependencies

| Package                             | Role                                           |
|-------------------------------------|------------------------------------------------|
| <code>anthropic</code>              | Claude API SDK                                 |
| <code>openai</code>                 | SDK used for DeepSeek/Groq (OpenAI-compatible) |
| <code>youtube-transcript-api</code> | Fetch YouTube captions                         |
| <code>yt-dlp</code>                 | Playlist metadata extraction                   |
| <code>reportlab</code>              | PDF generation                                 |
| <code>pyyaml</code>                 | YAML parsing                                   |
| <code>google-gemai</code>           | Google Gemini API SDK                          |

### 5.4 `patch.py` and `patch2.py` — API Migration Scripts

These scripts perform string-level code patching to migrate stages 2 and 3 from the OpenAI-compatible DeepSeek client to the Google Gemini API. `patch.py` migrates to `google.generativeai` (v0.x), while `patch2.py` migrates to the newer `google.gemai` SDK (v1.x) and upgrades the model from `gemini-1.5-pro` to `gemini-2.0-flash`. These are operational artifacts revealing the iterative development process of supporting multiple LLM back-ends.

## 6 LLM Integration Analysis

### 6.1 Supported Backends

The pipeline supports four distinct LLM backends:

| Backend       | Model                   | Cost                | Interface         |
|---------------|-------------------------|---------------------|-------------------|
| DeepSeek      | deepseek-chat           | Low                 | OpenAI-compatible |
| Anthropic     | claude-sonnet-4-...     | High                | Native SDK        |
| Groq          | llama-3.3-70b-versatile | Free (rate-limited) | OpenAI-compatible |
| Google Gemini | gemini-2.0-flash        | Low                 | Google GenAI SDK  |

### 6.2 Where the LLM is Used

The LLM is called in three distinct contexts:

1. **Chapter Introduction** (Stage 2): One call per video. Short prompt, 512 max tokens.
2. **Chunk Expansion** (Stage 2):  $k$  calls per video, where  $k = \lceil (|T| - O) / (C - O) \rceil$ . This is the dominant cost driver. 4096 max tokens per call.
3. **Foreword & Glossary** (Stage 3): Two calls total for the entire book. 1024 and 2048 max tokens respectively.

### 6.3 Cost Estimation

For a 43-video playlist with average transcript length  $|\bar{T}| = 3000$  characters:

$$k_i = \left\lceil \frac{|\bar{T}| - O}{C - O} \right\rceil = \left\lceil \frac{2800}{2800} \right\rceil = 1 \text{ chunk per video (approx.)}$$

In practice, longer videos (10,000+ chars) produce 3–4 chunks each. Total API calls  $\approx 43 \times 2 + 2 = 88$  minimum, up to 200+ for full playlist. At DeepSeek pricing ( $\approx \$0.001$  per call), the entire pipeline costs approximately \$0.10–\$0.20 USD.

### 6.4 Grounding Strategy

A key design decision is **strict transcript grounding**. The system prompt explicitly forbids the LLM from injecting external knowledge. This is a form of Retrieval-Augmented Generation (RAG) without a retrieval step: the context is directly injected rather than retrieved from a vector store.

## 7 Algorithms and Logic

### 7.1 Sliding-Window Chunking

Listing 1: Sliding-window text chunker with overlap

```

1 def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
2     chunks, start = [], 0
3     while start < len(text):
4         end = min(start + chunk_size, len(text))
5         chunk = text[start:end].strip()
6         if len(chunk) > 100:           # discard tiny trailing chunks

```

```

7         chunks.append(chunk)
8         next_start = start + chunk_size - overlap
9         if next_start <= start:             # guard against infinite loop
10             break
11         start = next_start
12     return chunks

```

The guard `next_start <= start` prevents an infinite loop when `chunk_size <= overlap`.

## 7.2 Exponential Backoff with Groq Rate-Limit Parsing

Listing 2: Smart rate-limit handler for Groq free tier

```

1 except RateLimitError as e:
2     m = re.search(r'try again in (?:(\d+)h)?(?:(\d+)m)?([\d\.]+)s', str(e))
3     if m:
4         h = int(m.group(1)) if m.group(1) else 0
5         m_min = int(m.group(2)) if m.group(2) else 0
6         s = float(m.group(3))
7         total_sleep = h * 3600 + m_min * 60 + s + 5 # +5s buffer
8         time.sleep(total_sleep)

```

## 7.3 Markdown-to-ReportLab Parser

The parser is a single-pass line scanner. It maintains state (`in_code_block`) and dispatches on line prefix patterns. Paragraph accumulation uses a lookahead inner loop:

Listing 3: Paragraph accumulation loop

```

1 para_lines = []
2 while i < len(lines) and lines[i].strip() and \
3     not lines[i].startswith(("#", "-", "'''", "---")):
4     para_lines.append(lines[i])
5     i += 1
6 if para_lines:
7     story.append(md_to_para(" ".join(para_lines), styles["body"]))

```

## 8 Design Patterns and Architecture Decisions

1. **Pipeline Pattern:** The four stages form a classic linear data pipeline where each stage's output is the next stage's input. The filesystem serves as the communication bus.
2. **Idempotency / Memoization:** File existence checks before every LLM call implement persistent memoization. This is equivalent to a disk-based cache keyed by `video_id`.
3. **Strategy Pattern (API Backends):** The `client` object is swappable between DeepSeek, Groq, Anthropic, and Gemini. The OpenAI SDK compatibility layer makes DeepSeek/Groq interchangeable without code changes.
4. **Template Method Pattern:** `robust_api_call` defines the retry skeleton, and the caller provides the specific API call logic.
5. **Separation of Concerns:** Content generation (Stage 2–3) is fully decoupled from rendering (Stage 4). Stage 4 has no LLM dependency.

6. **Configuration as Code:** All magic numbers (chunk size, overlap, model name) are externalized to `config.yaml`, enabling parameter sweeps without code changes.

## 9 Challenges and Limitations

---

### 9.1 Known Weaknesses

1. **Character-based chunking:** Chunking by character count rather than token count means actual token usage per chunk varies. A chunk of 3,000 characters of dense mathematical notation could use far more tokens than 3,000 characters of plain English.
2. **config.yaml mutation race condition:** `build_book.py` writes back to `config.yaml` to persist `max_videos`. Concurrent runs on the same repo would corrupt this file.
3. **No semantic chunking:** Chunks are split at arbitrary character positions, potentially cutting in the middle of a sentence, code block, or equation. Sentence-boundary-aware chunking would improve prose continuity.
4. **os.system() subprocess invocation:** Using `os.system` instead of `subprocess.run` provides no mechanism for capturing or redirecting stdout/stderr programmatically.
5. **Markdown parser is incomplete:** The custom parser does not handle nested lists, tables, blockquotes, or inline HTML. These may appear in LLM-generated output and would be rendered as raw text.
6. **No clickable TOC:** The PDF Table of Contents is not hyperlinked. ReportLab supports bookmarks and anchors, but these are not implemented.
7. **Glossary uses only first 8,000 characters:** The glossary is generated from only the first portion of the entire manuscript, potentially missing terms introduced in later chapters.
8. **Single-threaded execution:** Chapter generation is sequential. With 43 videos and 3 chunks each, this is  $\approx 130$  sequential API calls with 0.5s sleeps — processing time is dominated by API latency.

### 9.2 Edge Cases

- Videos with disabled transcripts (auto-captions off): Stage 1 catches the exception, logs `WARNING`, increments `failed` counter, and continues. No crash.
- Empty or very short transcripts ( $< 100$  chars): The chunker returns an empty list, and the chapter file will contain only the intro text. The stage handles this gracefully.
- Network timeout during LLM call: Caught by the generic `except Exception` clause in `robust_api_call`, triggering an exponential backoff retry.
- `config.yaml` missing at runtime: `load_config()` will raise a `FileNotFoundError` — no graceful error handling exists here.

### 9.3 Failure Scenarios

- **Persistent API failure:** If the LLM API is down indefinitely, `robust_api_call` will retry forever (no maximum retry limit).
- **Corrupted cached transcript:** If a `.json` file is partially written (e.g., interrupted mid-write), Stage 2 will fail with a JSON parse error.

- **ReportLab XML injection:** If an LLM generates text containing `<tag>` patterns (e.g., pseudo-HTML), the `md_to_para` function may produce malformed ReportLab XML, crashing Stage 4. The function escapes `&`, `<`, `>` but does so before applying bold/italic regex substitutions — the order matters and could be fragile.

## 10 Future Improvements

---

1. **Token-aware chunking:** Replace character-count chunking with a `tiktoken`-based tokenizer to ensure chunks stay within model context limits precisely.
2. **Parallel chapter generation:** Use `asyncio` or `concurrent.futures.ThreadPoolExecutor` to issue multiple LLM calls in parallel, reducing wall-clock time by  $N\times$  (where  $N$  is the parallelism factor).
3. **Semantic chunking:** Use sentence-boundary detection (`spaCy` or `NLTK sent_tokenize`) to split at sentence ends rather than arbitrary character offsets.
4. **Clickable PDF TOC:** Implement `ReportLab Bookmark` and internal link annotations for a navigable PDF.
5. **SQLite cache:** Replace file-based caching with a SQLite database for atomic writes, eliminating the partial-write corruption vulnerability.
6. **Multi-format output:** Add EPUB and DOCX export stages alongside PDF, using `ebooklib` and `python-docx` respectively.
7. **Quality scoring:** After generation, run an automated coherence check by sending the generated chapter back to the LLM and asking it to score factual consistency against the original transcript.
8. **Figure extraction:** Use `yt-dlp` to extract keyframes from the video at key timestamps and embed them as figures in the corresponding chapter.
9. **Vector RAG for glossary:** Instead of sending only the first 8,000 characters to the LLM for glossary generation, embed all chapters with a sentence transformer and query for the *top-k* most technical sentences across the full corpus.

## 11 Results and Output

---

The pipeline has been successfully run on the 43-video playlist. The outputs observed in the repository are:

- **43 transcript JSON files** in `cache/transcripts/`, totaling approximately 120,000 words of cleaned spoken content.
- **5 cached chapter Markdown files** in `free_api_version/cache/chapters/` (from test runs with `-max-videos 5`).
- `output/manuscript.md`: The full assembled manuscript for the 43-video run.
- `output/Building_LLMs_from_Scratch.pdf`: A publication-quality PDF with cover page, TOC, foreword, 43 chapters, and a glossary.
- Estimated final PDF: 200–350 pages, approximately 150,000–250,000 words of generated prose.

## 12 Conclusion

The `llm-book-pipeline` is a well-structured, practically motivated demonstration of LLM-powered content transformation. Its design exhibits sound software engineering principles: strict separation of concerns, idempotent stages, configurable parameters, and graceful failure handling. The prompt engineering is thoughtful — by grounding generation strictly in transcript content, the system avoids hallucination at the cost of relying entirely on source quality.

The most significant technical contributions of the project are (1) the robust retry and rate-limit handling that enables unattended free-tier API execution, (2) the custom ReportLab Markdown renderer that avoids LaTeX dependency entirely, and (3) the OpenAI SDK compatibility abstraction that allows four different LLM backends to be swapped with a single configuration change.

The key areas for improvement are parallelism (sequential API calls are a bottleneck), token-aware chunking (character-count chunking is imprecise), and a more robust caching mechanism to handle partial writes. Overall, the project is an excellent foundation for a production-grade educational content pipeline.

### Technical Summary Table

|                          |                                                        |
|--------------------------|--------------------------------------------------------|
| <b>Primary Language:</b> | Python 3.10+                                           |
| <b>LLM APIs Used:</b>    | DeepSeek, Anthropic, Groq, Google Gemini               |
| <b>Chunking:</b>         | Sliding window, 3000 chars, 200 char overlap           |
| <b>Prompt Pattern:</b>   | System + User, transcript-grounded, positional context |
| <b>PDF Engine:</b>       | ReportLab Platypus (no LaTeX)                          |
| <b>Caching:</b>          | File-based, keyed by <code>video_id</code>             |
| <b>Resumability:</b>     | Full — any stage can be restarted                      |
| <b>Parallelism:</b>      | None (sequential)                                      |
| <b>Estimated Cost:</b>   | \$0.10–\$0.20 USD (DeepSeek, full 43 videos)           |